



ULAB

UNIVERSITY OF LIBERAL ARTS
BANGLADESH

Project Proposal

Course code: [CSE3200](#)

Course Title: Design Project 1

Section: 3

Submitted by:

Zinnatun Tasnim (232014024)

Rafsan Janin washi (233014019)

Abdullah Al Tanmim (233014185)

Shihab Ahmed Sakib (231014148)

**AI-Powered Multi-Disease Chest X-Ray Triage and Clinical Decision Support
System**



CHEST X-RAY TRIAGE & CDSS

**AI-POWERED MULTI-DISEASE
CLINICAL DECISION SUPPORT SYSTEM
SUPPORTING CLINICAL CARE**

Introduction:

This project builds a Clinical Decision Support System for chest X-ray analysis. The system uses deep learning to detect multiple co-occurring thoracic conditions, such as Pneumonia, Tuberculosis, Cardiomegaly, and others. It pairs each prediction with a visual explanation showing which regions of the image influenced the result. That explainability layer matters, because a black-box confidence score isn't much use to a clinician who needs to know why. The deliverable is a mobile app with two distinct audiences. For patients, it gives an urgent triage assessment a way to know whether something needs immediate attention. For clinicians, it surfaces diagnostic suggestions mapped to standard care pathways, with the image evidence behind them. It does not prescribe. The system is designed to inform decisions, not replace them.

Problem Statement:

Deep learning has gotten genuinely good at reading chest X-rays in controlled settings, accuracy rivals trained radiologists. But those benchmark numbers haven't translated into tools that actually get used in clinics. Three problems explain most of that gap.

The first is that CNNs don't show their work. They take an image, run it through layers of learned filters, and output a probability. There's no accompanying explanation no highlighted region, no reasoning chain. In a legal and ethical sense, that's a non-starter for clinical use. A clinician can't act on "87% probability of Pneumonia" if they have no way to interrogate or challenge it. Trust requires transparency, and transparency is exactly what most models skip.

The second problem is that real patients are messier than training datasets. Someone admitted with breathlessness might have Pneumonia sitting on top of an already enlarged heart. Multi-label classification detecting several co-occurring conditions at once is harder than it looks, especially when the underlying dataset has far more examples of common diseases than rare ones. Models trained this way develop a bias: they're reasonably good at catching what's common and quietly miss what isn't. That's a real clinical risk.

The third is a gap between output and action. A probability score is not a triage decision. Right now, most systems stop at the inference step they generate a finding and hand it off, and whatever happens next is someone else's problem. No

pipeline takes a model's output and converts it into a clear urgency flag for a patient, or a structured diagnostic prompt for a doctor, while keeping the system from overstepping into autonomous prescription.

Objective:

- **Develop a Multi-Label Deep Learning Architecture:** Fine-tune state-of-the-art pre-trained Convolutional Neural Networks (e.g., DenseNet121, ResNet50) to accurately identify co-occurring thoracic pathologies (such as Pneumonia, Tuberculosis, and Cardiomegaly) while implementing techniques to mitigate the inherent class imbalances found in large-scale medical datasets.
- **Implement Algorithmic Transparency via XAI:** Integrate Explainable AI techniques, specifically Gradient-weighted Class Activation Mapping (Grad-CAM), to generate high-resolution spatial heatmaps that visually isolate the anatomical regions driving the model's predictions, thereby transforming the "black box" model into a clinically verifiable tool.
- **Engineer a Cross-Platform Application Interface:** Develop a secure, user-friendly mobile application and a robust backend API (using Python frameworks) that facilitates seamless X-ray image uploads, real-time deep learning inference, and the immediate return of diagnostic data and visual evidence.
- **Construct a Rule-Based Clinical Triage Engine:** Design a deterministic logic layer that translates the AI's probabilistic outputs into safe, actionable guidance. This includes generating urgency-based "emergency care" alerts for patient-users and displaying standardized, evidence-based treatment pathways for clinician-users, strictly acting as a decision-support mechanism rather than an autonomous prescriber.
- **Evaluate System Performance and Usability:** Rigorously validate the machine learning model using standard quantitative metrics (Precision, Recall, F1-Score, and AUC-ROC) and assess the end-to-end software system for latency, scalability, and clinical user interface efficacy.

1. Why would a professional doctor use this app and focus on it and suggest to the patients?

By functioning as a tireless digital assistant, this application seamlessly enhances both a doctor's diagnostic accuracy and their patient communication. Behind the scenes, the AI pre-screens X-rays to instantly flag critical emergencies and acts as a fatigue-proof "second reader" to catch subtle, overlapping conditions that a tired physician might overlook. During consultations, the doctor can use the app's Explainable AI (XAI) heatmaps as a direct visual aid, translating complex grayscale images into clear, color-coded evidence. This collaborative workflow allows the doctor to easily explain the diagnosis, validate their treatment plan, and build the immediate patient trust necessary for high treatment compliance.

Proposed Methodology:

Phase 1: Data Acquisition and Preprocessing Layer

1. Database Selection: The system utilizes large-scale, open-source medical imaging databases such as the NIH ChestX-ray14 or CheXpert dataset. These datasets contain hundreds of thousands of high-resolution frontal-view chest X-ray scans mapped to multi-label ground truth annotations.
2. Image Optimization & Enhancement: Raw X-rays frequently exhibit varying contrast levels due to different hospital equipment. To standardize inputs, Contrast Limited Adaptive Histogram Equalization (CLAHE) is applied to balance local contrast without amplifying image noise. Scans are then downsampled to 224×224 or 512×512 pixels to optimize computational performance during training.
3. Data Augmentation: To mitigate severe class imbalances (e.g., significantly fewer instances of Pneumothorax compared to Normal lungs), data expansion techniques—including random rotations ($\pm 10^\circ$), horizontal flipping, and minor scaling—are introduced exclusively to the training set to prevent model overfitting.

Phase 2: Multi-Label Deep Learning Architectural Pipeline

1. **Transfer Learning Backbone:** Given the limited computational resources compared to massive tech enterprises, training a deep network from scratch is impractical. The system leverages a pre-trained DenseNet121 or ResNet50 architecture, originally trained on ImageNet, utilizing its pre-existing feature extraction capabilities.
2. **Network Customization:** The final dense classification layer of the network is removed and replaced with a fully connected layer matching the exact number of target thoracic diseases in your dataset.
3. **Activation and Optimization:** Because a single chest X-ray can present multiple overlapping pathologies simultaneously, standard Softmax activation cannot be used. Instead, a Sigmoid activation function is applied to each output node independently, treating the system as a collection of concurrent binary classification problems. The network optimizes using a weighted Binary Cross-Entropy (BCE) Loss Function to penalize misclassifications across all disease labels equally.

Phase 3: Explainable AI (XAI) Interpretation Layer

To transform the model from an opaque "black box" into a verifiable clinical assistant, an interpretability layer is integrated into the inference cycle:

1. **Grad-CAM Localization:** Gradient-weighted Class Activation Mapping (Grad-CAM) calculates the gradients of the score for a specific disease class with respect to the feature map activations of the very last convolutional layer in the DenseNet backbone.
2. **Saliency Mapping:** These gradients are globally pooled to determine the importance weight of each feature map. A rectified linear combination of the maps is then computed to isolate the exact spatial coordinates (pixels) that positively influenced the AI's final probability score.
3. **Heatmap Generation:** This low-resolution activation map is upsampled to match the original X-ray dimensions and converted into a color-coded alpha-blended heatmap, overlaying the suspected lesion areas for physician verification.

Phase 4: Full-Stack System Integration & Application Architecture

The production phase materializes the model into a responsive mobile software environment via a decoupled, asynchronous architecture:

1. The Backend API Layer: Constructed using high-performance Python frameworks (FastAPI or Flask), the backend hosts the trained deep learning weights. It exposes a secure REST API endpoint (/predict) designed to handle incoming multipart form-data (image uploads).
2. The Mobile Client Interface: Developed using cross-platform frameworks, the application provides two tailored user experiences:
 - *Patient View*: Allows users to upload/scan an image and instantly receive emergency response instructions based on a rule-based triage logic engine mapped to the AI's output severity.
 - *Clinician View*: Displays a secure diagnostic dashboard containing the multi-label class probabilities, the generated XAI heatmaps confirming visual focus, and standard pharmaceutical reference pathways for immediate professional review.

Expected Machine Pipeline:

1. Data Ingestion & Preprocessing (The Input Layer)

Medical datasets like CheXpert or NIH ChestX-ray14 are massive and highly imbalanced. The pipeline must first standardize these inputs before feeding them to the neural network.

- **Image Standardization**: X-rays come in different resolutions and contrast levels. The Python pipeline will use OpenCV or PIL to resize all images to a uniform dimension (e.g., $224 \times 224 \times 3$).
- **Contrast Enhancement**: To ensure the model doesn't fail on washed-out X-rays, Contrast Limited Adaptive Histogram Equalization (CLAHE) is applied to enhance the definition of lung structures and subtle lesions.
- **Data Augmentation**: To prevent the model from overfitting on common diseases while ignoring rare ones, the training set undergoes dynamic transformations (minor rotations, horizontal flips, and brightness

adjustments) using PyTorch's `torchvision.transforms` or TensorFlow's `ImageDataGenerator`.

2. Architecture & Transfer Learning (The Deep Feature Layer)

Instead of building a Convolutional Neural Network from scratch, the pipeline utilizes transfer learning to achieve high accuracy with less computational overhead.

- **The Backbone:** A pre-trained model like DenseNet121 is loaded. DenseNets are particularly effective for medical imaging because their dense connectivity pattern ensures that low-level features (like edges and textures, crucial for identifying lung opacities) are passed directly to the final classification layers without vanishing.
- **Modifying the Classification Head:** The original 1000-class ImageNet fully connected layer is stripped off. It is replaced with a custom Dense layer where the number of output nodes matches your specific target diseases (e.g., 14 nodes for 14 diseases).
- **Multi-Label Activation:** Because a patient can have Pneumonia and Cardiomegaly simultaneously, a Softmax activation cannot be used. Instead, a Sigmoid activation function is applied to each of the output nodes, outputting an independent probability between 0 and 1 for every single disease.

3. Training & Optimization (The Learning Layer)

- **Loss Function:** The model evaluates its own errors using Binary Cross-Entropy (BCE) Loss. This calculates the error for each disease class independently and averages it out.
- **Optimizer:** The Adam optimizer is utilized for its adaptive learning rate, ensuring the network converges efficiently without getting stuck in local minima.
- **Class Weighting:** Because "Normal" scans outnumber "Pneumothorax" scans 10-to-1 in most databases, the loss function is penalized more heavily when it misclassifies a rare disease, forcing the network to pay attention to underrepresented conditions.

4. Interpretability & XAI (The Transparency Layer)

This is where the "black box" is opened, validating the AI for clinical use.

- Gradient Extraction: During inference, the pipeline isolates the final convolutional layer of the DenseNet model. It calculates the gradients (the "attention") of the target disease score with respect to the feature maps in this final layer.
- Grad-CAM Heatmap Generation: By pooling these gradients, the pipeline identifies which specific spatial regions (pixels) of the X-ray contributed most to the positive classification.
- Overlay & Output: This spatial map is upsampled, colored (usually using a JET colormap where red equals high importance), and overlaid onto the original grayscale X-ray.

System Architecture:

The Backend Microservice (FastAPI & Python)

The backend acts as the computational engine of the application. It is constructed using FastAPI, a modern, high-performance web framework for Python. FastAPI is chosen specifically over Django or Flask because of its native support for asynchronous request handling and its ability to serve heavy machine learning models efficiently without blocking user traffic.

- RESTful API Endpoints: The server exposes secure endpoints (e.g., POST /api/v1/predict/chest-xray) designed to accept multipart form-data (the raw X-ray image) from the mobile client.
- Model Serving & Inference: Upon receiving an image, the FastAPI server temporarily loads the scan into memory, applies the required contrast enhancements (CLAHE) and resizing, and passes it through the fine-tuned DenseNet/ResNet PyTorch model.
- Dynamic XAI Generation: Post-inference, the server runs the Grad-CAM script to generate the spatial heatmap. Using OpenCV, it blends the colored heatmap over the original grayscale X-ray, encodes the final image as a

base64 string, and packages it with the multi-label probability JSON object to send back to the user.

The Frontend Mobile Interface (Flutter & Dart)

To maximize accessibility across both Android and iOS devices using a single codebase, the frontend application is developed using Flutter. Flutter's Skia rendering engine ensures that the heavy, high-resolution medical images and XAI heatmaps load smoothly without UI lag.

The application features a dual-interface design mapped to user roles:

- The Patient Interface (Self-Triage): Designed for simplicity. Users upload a scan from their phone gallery or camera. The UI strips away complex medical statistics, instead displaying a clear, color-coded triage status (e.g., *Red: Seek Immediate Care, Green: Routine Checkup Recommended*) based on the backend logic.
- The Clinician Interface (Diagnostic Dashboard): Designed for data depth. Doctors securely log in to view patient queues. Upon opening a scan, they are presented with the original X-ray, a toggle to overlay the Grad-CAM heatmap, and a detailed breakdown of the model's multi-label percentage scores across all 14 thoracic disease classes.

The Clinical Logic & Triage Engine

The raw probability scores generated by the AI (e.g., *Pneumothorax: 0.92, Pneumonia: 0.85*) are clinically meaningless without context. The backend includes a deterministic, rule-based logic engine that acts as a bridge between data science and medical practice.

- Urgency Thresholds: The engine evaluates the AI's output against predefined severity thresholds. If a critical condition (like Pneumothorax) crosses the 85% probability mark, the engine appends a *"severity": "CRITICAL"* flag to the JSON response, triggering the mobile app's emergency red-alert UI.
- Care Pathway Mapping: For the clinician dashboard, the engine maps the detected diseases to standard pharmaceutical or diagnostic guidelines, displaying a brief text prompt (e.g., *"Suggest secondary CT scan for verified nodule presence"*), strictly acting as decision support.

Security, Compliance, and Deployment

- **Stateless Processing:** To respect patient privacy, the FastAPI backend processes the images in memory and immediately drops the data post-inference. No medical scans are permanently stored on the server database unless explicitly saved by the physician for training purposes.
- **Containerization:** The entire backend API, including the PyTorch environment and model weights, is containerized using Docker. This ensures that the deep learning environment remains completely consistent whether it is running on a local testing laptop or deployed to a cloud provider like AWS or Google Cloud.

System Interaction Flow

1. Client (Flutter): User uploads X-ray, Encodes image, Sends HTTP POST request.
2. Server (FastAPI): Validates request Preprocesses image array Executes model inference.
3. Inference (PyTorch): Calculates 14 multi-label probabilities Extracts Grad-CAM gradients Generates heatmap overlay.
4. Logic Engine: Evaluates thresholds Assigns "Triage Status".
5. Server (FastAPI): Packages JSON (Probabilities, Triage Status, Base64 Heatmap) Returns HTTP 200 Response.
6. Client (Flutter): Decodes payload Renders Dashboard UI.

Conclusion:

Deep learning has gotten genuinely good at reading chest X-rays. In controlled benchmark settings, accuracy rivals trained radiologists. That fact gets cited a lot. What gets cited less is that these models have barely moved into actual clinical use and the gap between a strong paper and a deployed tool is wider than it looks. Three things explain most of it. CNNs don't show their work. An image goes in, a probability comes out, and the reasoning in between is opaque. That's a practical

problem, not just a philosophical one. A clinician who can't interrogate a prediction can't responsibly act on it legally or ethically. Accuracy alone doesn't build trust. Transparency does, and most models don't provide it. Real patients are also messier than training data. Someone admitted with breathlessness might have Pneumonia sitting on top of an already enlarged heart. Catching both, simultaneously, from a single image is harder than the single-label benchmarks suggest. It gets harder still when the training dataset has seen ten thousand cases of one condition and a few hundred of another. Models learn that imbalance and quietly carry it forward performing well on common presentations and missing the rarer co-occurring ones that arguably matter more. Then there's the gap between a model output and something a person can actually use. A probability score isn't a triage decision. Most systems stop at inference, they generate a finding, and whatever happens next is left to someone else. There's no pipeline that takes that output and converts it into a clear urgency signal for a patient, or a structured clinical prompt for a doctor without the system crossing into making the call itself.